

搜索

Meaninglessness

西安交通大学

July 9, 2026



目录

- 1 基本概念
- 2 深度优先搜索
- 3 广度优先搜索
- 4 迭代加深
- 5 meet in the middle
- 6 剪枝
- 7 其它应用
- 8 习题

目录

- 1 基本概念
- 2 深度优先搜索
- 3 广度优先搜索
- 4 迭代加深
- 5 meet in the middle
- 6 剪枝
- 7 其它应用
- 8 习题

基本概念

搜索，也就是对状态空间进行枚举，通过穷尽所有的可能来找到最优解，或者统计合法解的个数。

搜索算法实际上是根据初始条件和扩展规则构造一棵“搜索树”并寻找符合目标状态的节点的过程。搜索算法的使用第一步在于搜索树的建立，完成搜索的过程就是找到一条从根结点到目标结点的路径，类似于图或树的遍历。

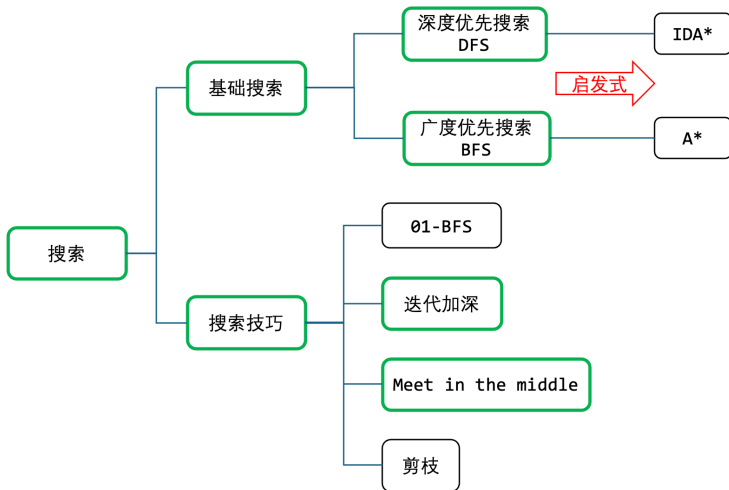
关于搜索的复杂度：目标状态总数 $\times$ 目标状态处理

求解过程

搜索算法的构建思路：

- 1 确定搜索中途状态与目标状态
- 2 如何从一个已知状态扩展至未知状态
- 3 到达一个目标状态后的处理
- 4 必要的优化剪枝

知识框架



目录

1 基本概念

2 深度优先搜索

3 广度优先搜索

4 迭代加深

5 meet in the middle

6 剪枝

7 其它应用

8 习题

递归

- 递归，是指在程序中出现**函数调用自身**的现象。
- 其思想是将原问题转化为更小规模的子问题，当子问题规模足够小，可以直接计算答案时，则终止递归。
- 注意：没有递归边界的递归函数会死循环！
- 因此递归时，我们要明确两点：原问题转化为子问题的公式是什么？递归边界是什么？
- 示例：用递归函数的方法求斐波那契数列的第 n 项 fib_n 。

递归

- 递归，是指在程序中出现**函数调用自身**的现象。
- 其思想是将原问题转化为更小规模的子问题，当子问题规模足够小，可以直接计算答案时，则终止递归。
- 注意：没有递归边界的递归函数会死循环！
- 因此递归时，我们要明确两点：原问题转化为子问题的公式是什么？递归边界是什么？
- 示例：用递归函数的方法求斐波那契数列的第 n 项 fib_n 。

$$\text{fib}_n = \begin{cases} 1 & n = 1, 2 \\ \text{fib}_{n-1} + \text{fib}_{n-2} & n \geq 3 \end{cases}$$

递归

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int fib(int n) {
5      if(n == 1 || n == 2) return 1;
6      return fib(n - 1) + fib(n - 2);
7  }
8
9  int main() {
10     int n;
11     cin >> n;
12     cout << fib(n) << endl;
13     return 0;
14 }
```

算法描述

通过递归函数，来保证每次都尝试往更深的节点搜索，如果没有更深状态或已经搜索完更深状态，就会回溯至上一状态。

```
1 void dfs(状态集 S)
2     if(S 为目标状态) {
3         处理目标状态 S
4         return;
5     }
6     // 此时还不是目标状态
7     for(由 S 一步生成的未知状态 T) {
8         if(状态 T 合法) {
9             更新相关信息
10            dfs(T);
11            必要的回溯操作处理
12        }
13    }
14    // 对于 S 状态，比它深层的状态已经都遍历完了
15    // 这里由于递归函数的特点，会自动回溯到 S 的上一个状态
16 }
```

例题 1：全排列问题

按照字典序升序输出自然数 1 到 n 所有不重复的排列，即 n 的全排列，要求所产生的任一数字序列中不允许出现重复的数字。 $1 \leq n \leq 9$ 。

输入 #1

3

输出 #1

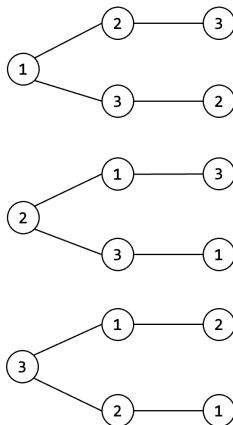
1	2	3
1	3	2
2	1	3
2	3	1
3	1	2
3	2	1

例题 1：全排列问题

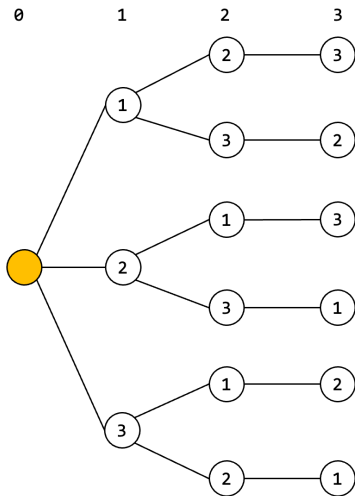
作为一个人类，你用什么方法数全排列？

例题 1：全排列问题

作为一个人类，你用什么方法数全排列？



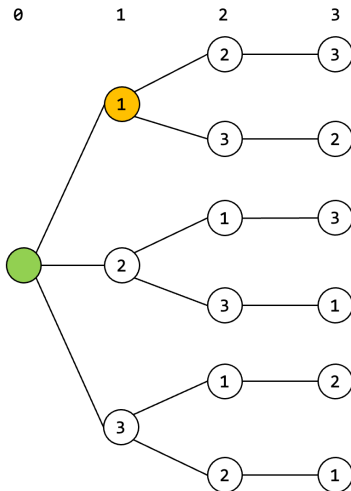
例题 1：全排列问题



dep = 0

a[] = {}

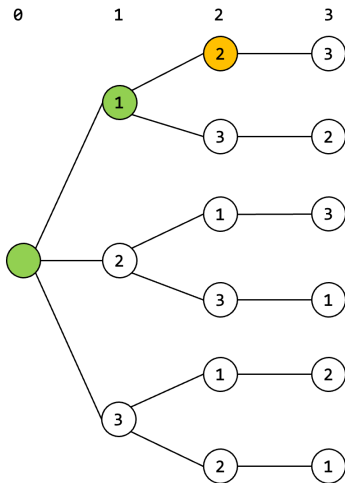
例题 1：全排列问题



dep = 1

a[] = {1}

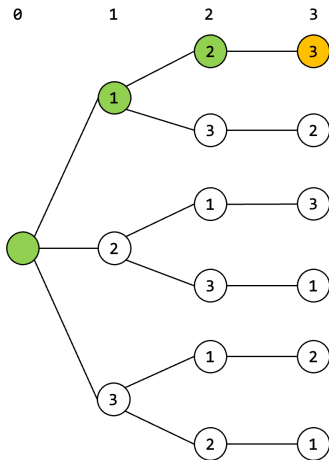
例题 1：全排列问题



dep = 2

a[] = {1, 2}

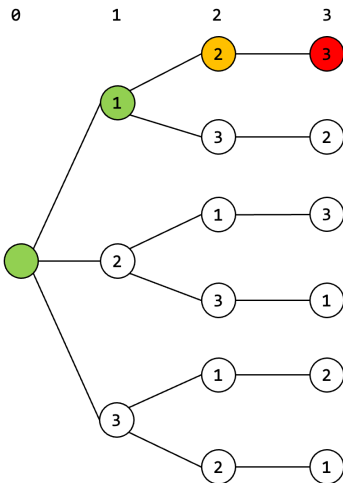
例题 1：全排列问题



dep = 3

a[] = {1, 2, 3}

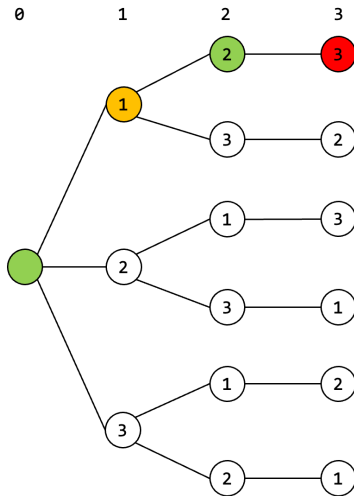
例题 1：全排列问题



dep = 2

a[] = {1, 2}

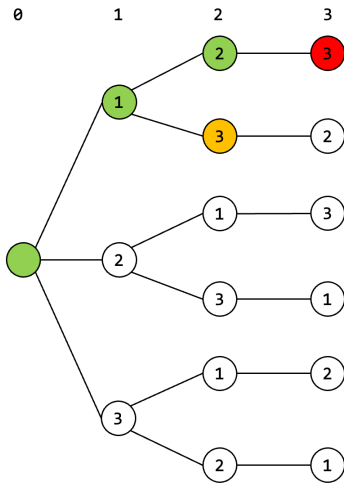
例题 1：全排列问题



dep = 1

a[] = {1}

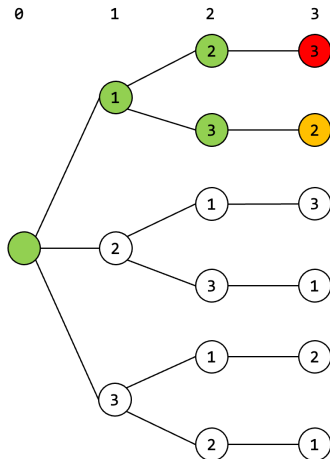
例题 1：全排列问题



dep = 2

a[] = {1, 3}

例题 1：全排列问题



dep = 3

a[] = {1, 3, 2}

例题 1：全排列问题

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  int n, a[10];
4  bool vis[10];
5
6  void dfs(int u) {
7      if(u == n) {
8          for(int i = 0; i < n; ++i)
9              cout << " " << a[i];
10             cout << '\n';
11             return;
12         }
13         for(int i = 1; i <= n; ++i) {
14             if(!vis[i]) {
15                 vis[i] = true, a[u] = i;
16                 dfs(u + 1);
17                 vis[i] = false;
18             }
19         }
20     }
21
22     int main() {
23         cin >> n; dfs(0);
24         return 0;
25     }
```

例题 2：迷宫

给定一个 $N \times M$ 方格的迷宫，迷宫里有 T 处障碍，障碍处不可通过。

在迷宫中移动有上下左右四种方式，每次只能移动一个方格。数据保证起点上没有障碍。

给定起点坐标和终点坐标，每个方格最多经过一次，问有多少种从起点坐标到终点坐标的方案。

对于 100% 的数据， $1 \leq N, M \leq 5$ ， $1 \leq T \leq 10$ ，保证障碍坐标在迷宫范围内。

输入 #1

```
2 2 1
1 1 2 2
1 2
```

输出 #1

```
1
```

例题 2: 迷宫

- 1 状态：目前所处的坐标 (x, y)
- 2 扩展：向上下左右四个方向移动，并判断是否合法
- 3 起始：起点坐标
- 4 目标：终点坐标

例题 2：迷宫

- 1 状态：目前所处的坐标 (x, y)
- 2 扩展：向上下左右四个方向移动，并判断是否合法
- 3 起始：起点坐标
- 4 目标：终点坐标

时间复杂度： $O(\text{合法路径数})$

例题 2：迷宫

```
1  const int N = 15;
2  const int dx[4] = {0, 0, +1, -1}, dy[4] = {+1, -1, 0, 0};
3  int n, m, t, sx, sy, fx, fy, ans;
4  bool block[N][N], vis[N][N];
5
6  void dfs(int x, int y) {
7      if(x == fx && y == fy) {
8          ++ans; return;
9      }
10     for(int i = 0; i < 4; ++i) {
11         int nx = x + dx[i], ny = y + dy[i];
12         if(nx >= 1 && nx <= n && ny >= 1 && ny <= m && !block[nx][ny] && !vis[nx][ny]) {
13             vis[nx][ny] = true;
14             dfs(nx, ny);
15             vis[nx][ny] = false;
16         }
17     }
18 }
19
20 int main() {
21     cin >> n >> m >> t >> sx >> sy >> fx >> fy;
22     while(t--) {
23         int x, y;
24         cin >> x >> y;
25         block[x][y] = true;
26     }
27     vis[sx][sy] = true; dfs(sx, sy);
28     cout << ans << '\n';
29     return 0;
30 }
```



目录

1 基本概念

2 深度优先搜索

3 广度优先搜索

4 迭代加深

5 meet in the middle

6 剪枝

7 其它应用

8 习题

广度优先搜索

- 广度优先搜索 (Breadth First Search)，即在搜索的过程中以**广度**为第一优先级进行考虑。
- 每次尝试访问同一层的节点，当同一层节点访问完后，再访问下一层。
- BFS 算法找到的路径是从起点开始的**最短**合法路径。
- “火势蔓延”：从起点开始，距离近的先被烧到。
- 通常使用**队列**进行实现。

队列

- 先进先出（FIFO）的数据结构
- 我们可以借助 STL 中的 `queue` 来帮助我们实现队列的基本操作，包括插入元素，取队首，查询队列中元素个数等。

算法描述

使用**队列**以保持遍历过的节点层序。

```
1 void bfs() {  
2     定义队列  
3     将起点加入队列，并标记为访问  
4     while(队列非空) {  
5         取出队首元素 u  
6         for(u 的邻接状态 v) {  
7             if(v 没有被访问过) {  
8                 处理 u 与 v 的关系  
9                 将 v 加入队列，并标记为访问  
10            }  
11        }  
12    }  
13 }
```

例题 3：马的遍历

有一个 $n \times m$ 的棋盘，在某个点 (x, y) 上有一个马，要求你计算出马到达棋盘上任意一个点最少要走几步。不能到达的点输出 -1 。

保证 $1 \leq x \leq n \leq 400, 1 \leq y \leq m \leq 400$ 。

输入 #1

3 3 1 1

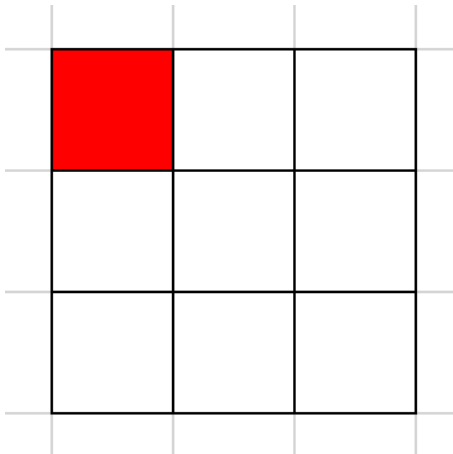
输出 #1

0 3 2

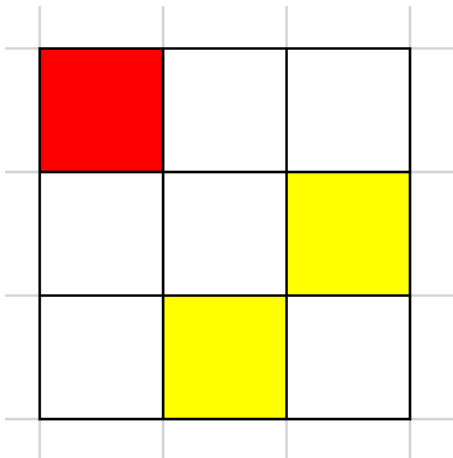
3 -1 1

2 1 4

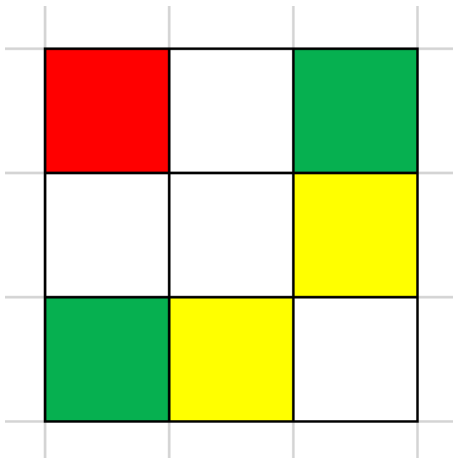
例题 3：马的遍历



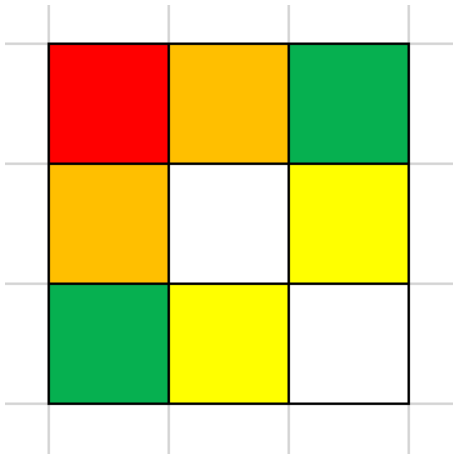
例题 3：马的遍历



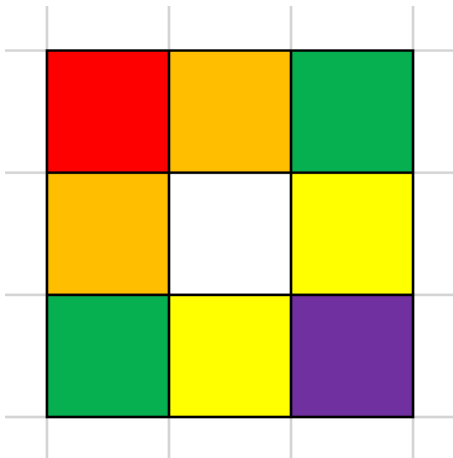
例题 3：马的遍历



例题 3：马的遍历



例题 3：马的遍历



例题 3：马的遍历

把坐标视为状态，那么就是求每个点到起点的最短路径，可以用 **BFS** 求解。

时间复杂度： $\mathcal{O}(nm)$

例题 3：马的遍历

```
1  const int N = 405;
2  const int dx[8] = {+1, +2, +2, +1, -1, -2, -2, -1};
3  const int dy[8] = {+2, +1, -1, -2, -2, -1, +1, +2};
4  int n, m, x, y;
5  bool vis[N][N];
6  int step[N][N];
7
8  void bfs() {
9      queue<int> que;
10     que.push(x); que.push(y);
11     vis[x][y] = true, step[x][y] = 0;
12     while(!que.empty()) {
13         int x1 = que.front(); que.pop();
14         int y1 = que.front(); que.pop();
15         for(int i = 0; i < 8; ++i) {
16             int x2 = x1 + dx[i], y2 = y1 + dy[i];
17             if(x2 >= 1 && x2 <= n && y2 >= 1 && y2 <= m && !vis[x2][y2]) {
18                 vis[x2][y2] = true, step[x2][y2] = step[x1][y1] + 1;
19                 que.push(x2); que.push(y2);
20             }
21         }
22     }
23 }
```

例题 3：马的遍历

```
1  int main() {
2      cin >> n >> m >> x >> y;
3      memset(step, -1, sizeof step);
4      bfs();
5      for(int i = 1; i <= n; ++i) {
6          for(int j = 1; j <= m; ++j) {
7              cout << step[i][j] << 'u';
8          }
9          cout << '\n';
10     }
11     return 0;
12 }
```

目录

1 基本概念

2 深度优先搜索

3 广度优先搜索

4 迭代加深

5 meet in the middle

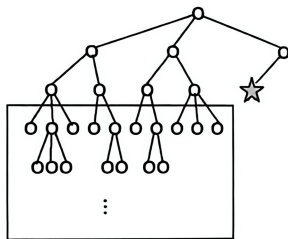
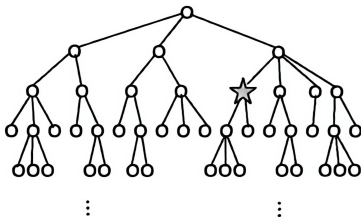
6 剪枝

7 其它应用

8 习题

迭代加深

传统的 DFS 有何缺点？

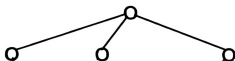


迭代加深

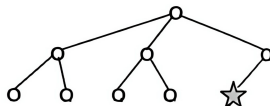
- 迭代加深的本质是**带深度限制**的深度优先搜索。
- 在搜索的同时带上了一个深度，当达到设定的深度时就返回，一般用于找最优解。如果一次搜索没有找到合法的解，就让设定的深度加一，重新从根开始。
- 相较于 **BFS**，迭代加深不用使用队列存储大量的状态，空间复杂度相对较小。因此也可以看作是空间优化版的 **BFS**。



限制搜索深度为1
搜索失败



限制搜索深度为2
搜索失败



限制搜索深度为3
找到答案

算法描述

```
1 void dfs(状态集 S, int dep, int maxdep) {
2     if(dep > maxdep)
3         return;
4     if(S 为目标状态) {
5         处理目标状态 S
6         return;
7     }
8     // 此时还不是目标状态
9     for(由 S 一步生成的未知状态 T) {
10        if(状态 T 合法) {
11            更新相关信息
12            dfs(T, dep + 1, maxdep);
13            必要的回溯操作处理
14        }
15    }
16 }
17
18 void solve() {
19     for(int maxdep = 1; ; ++maxdep) {
20         dfs(初始状态, 0, maxdep);
21         if(得到答案)
22             break;
23     }
24 }
```

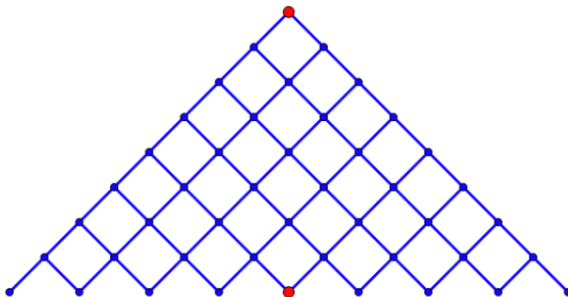
目录

- 1 基本概念
- 2 深度优先搜索
- 3 广度优先搜索
- 4 迭代加深
- 5 meet in the middle**
- 6 剪枝
- 7 其它应用
- 8 习题

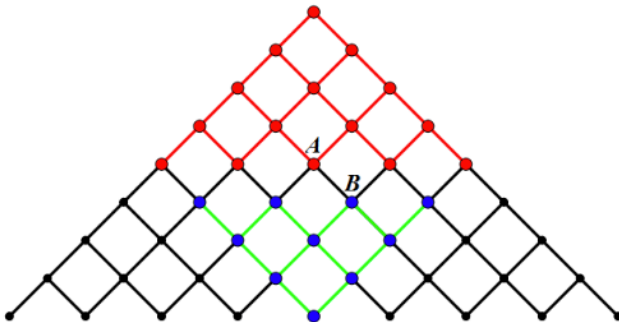
meet in the middle

- **meet in the middle** (中文翻译为「折半搜索」), 其主要思想是将整个搜索过程分为两半, 分别搜索, 最后将两部分的结果合并。
- 它适用于输入数据较小, 但还没小到能直接使用暴力搜索的情况。

meet in the middle



meet in the middle



例题 4: [USACO2.2] 集合 Subset Sums

对于从 $1 \sim n$ 的连续整数集合，能划分成两个子集合，且保证每个集合的数字和是相等的。

例如 $n = 7$ ，有四种方法能划分集合 $\{1, 2, 3, 4, 5, 6, 7\}$ ：

- $\{1, 6, 7\}$ 和 $\{2, 4, 3, 5\}$
- $\{2, 5, 7\}$ 和 $\{1, 3, 4, 6\}$
- $\{3, 4, 7\}$ 和 $\{1, 2, 5, 6\}$
- $\{1, 2, 4, 7\}$ 和 $\{3, 5, 6\}$

现给定 n ，求出划分的方案总数。 $1 \leq n \leq 39$ 。

输入 #1

7

输出 #1

4

例题 4: [USACO2.2] 集合 Subset Sums

注意到划分为两个相等的子集合，等价于每一个集合的数字和都是 $m = \frac{n(n+1)}{2}$ 的一半。

那么我们考虑计算：满足和为 $\frac{m}{2}$ 的 $1 \sim n$ 的子集数量。

直接搜索是 $\mathcal{O}(2^n)$ 的，不能接受。考虑将这 n 个数拆成两半，前一半和后一半的方案分别存在数组 `cnt1` 和 `cnt2` 中，则所求子集数量为：

$$\sum_{i=0}^{\frac{m}{2}} \text{cnt1}_i \times \text{cnt2}_{\frac{m}{2}-i}$$

由于一组划分方案对应了两个子集，需要对上式除以二才是最终的答案。

此时时间复杂度优化至 $\mathcal{O}(2^{\frac{n}{2}})$ ，可以通过。

当然本题还有 $\mathcal{O}(n^2)$ 的动态规划做法，这里不作展开。

例题 4: [USACO2.2] 集合 Subset Sums

```
1  typedef long long ll;
2  const int N = 2005;
3  int n, m;
4  ll cnt1[N], cnt2[N], ans;
5
6  void dfs(int u, int tot, int limit, ll cnt[]) {
7      if(u > limit) {
8          ++cnt[tot];
9          return;
10     }
11     dfs(u + 1, tot, limit, cnt);
12     dfs(u + 1, tot + u, limit, cnt);
13 }
14
15 int main() {
16     cin >> n;
17     m = n * (n + 1) / 2;
18     if(m % 2 != 0) {
19         cout << "0\n";
20         return 0;
21     }
22     dfs(1, 0, n / 2, cnt1);
23     dfs(n / 2 + 1, 0, n, cnt2);
24     for(int i = 0; i <= m / 2; ++i)
25         ans += cnt1[i] * cnt2[m / 2 - i];
26     cout << ans / 2 << endl;
27     return 0;
28 }
```

目录

- 1 基本概念
- 2 深度优先搜索
- 3 广度优先搜索
- 4 迭代加深
- 5 meet in the middle
- 6 剪枝**
- 7 其它应用
- 8 习题

剪枝

- 通俗地讲，剪枝就是提前排除一些不必要的搜索路径。由于这个过程很像在一棵树上剪掉一些“枝条”，故称为剪枝。
- 常见的几种优化策略：
 - 1 可行性：当前的状态无论如何都不会到达目标状态（走入死胡同），那么直接回溯；
 - 2 最优性：在最优化问题中，如果可以说明当前状态已经劣于已到达的目标状态的答案，则也直接回溯
- 剪枝优化往往比较玄学，强依赖于具体题目，大家在做搜索题目时可以多加练习。

目录

- 1 基本概念
- 2 深度优先搜索
- 3 广度优先搜索
- 4 迭代加深
- 5 meet in the middle
- 6 剪枝
- 7 其它应用**
- 8 习题

其它应用

一般在 **ACM** 的正式赛中，很少出现**直接使用**搜索算法通过的题目。

考虑搜索算法的特点：暴力直观，一般情况下代码好写，且保证正确。

那么我们在比赛时，可以通过写“对拍”的方式来检查我们的正解。我们要写两份代码，一份是搜索实现的朴素暴力，一份是不保证正确的预期正解，然后通过数据生成器生成一组数据，让两份代码分别运行，并比较得到的结果是否一致。

基础教程：<https://www.luogu.com.cn/article/uuh1asja>

目录

- 1 基本概念
- 2 深度优先搜索
- 3 广度优先搜索
- 4 迭代加深
- 5 meet in the middle
- 6 剪枝
- 7 其它应用
- 8 习题**

鸣谢

- 感谢西安交通大学算法竞赛校队以及“这是一个 **ACM** 队”中的同学支持。
- 参考文献：往年课件、OI-wiki、《算法竞赛进阶指南》等。

Thanks for listening.